

Volatility Targeting for ETF Allocation

An Independent Student Research Note

Ben Mellors
December 2025

Abstract

This student research note explores volatility targeting as a systematic portfolio construction method for ETF allocation. Using historical data, the paper develops, tests, and evaluates volatility-scaled strategies against buy-and-hold benchmarks, focusing on risk-adjusted performance.

1 Introduction

This note documents a build of a beginner-friendly systematic investing research workflow in Python. Starting from a fresh setup (VS Code, Python, packages), I implemented a small backtesting engine using daily ETF data and evaluated strategies using total return, Sharpe ratio, and maximum drawdown. Early timing rules (for example moving average crossovers) generally underperformed a buy-and-hold benchmark once basic frictions were added. The best performing unlevered model in this project is a volatility-targeted allocation across two equity ETFs (SPY and QQQ), which achieved higher total return, higher Sharpe, and smaller drawdown than SPY buy-and-hold over the tested period.

This is an independent, self-taught research exercise (not university coursework). However, studying mathematics (in particular probability, statistics, and time series ideas) helped with understanding the modelling choices and evaluation metrics.

2 What this project is (in plain English)

The goal was not to “build a hedge fund in a day”. The goal was to build a repeatable research process:

- choose a market universe (ETFs),
- define rules (signals and position sizing),
- simulate trades with basic realism (costs, cash, rebalancing),
- measure performance (return, Sharpe, drawdown),
- iterate and document results.

This research note is written to be readable by someone new to quant finance and coding.

3 Beginner definitions

Asset / ETF: An exchange-traded fund (ETF) is a basket of investments that trades like a stock. SPY tracks the S&P 500 (large US equities). QQQ tracks the Nasdaq-100 (tech-heavy US equities).

Benchmark: The simple baseline you compare against (for example buy-and-hold SPY).

Return multiple: If you start with £1 and end with £4, that is a 4.00x return multiple.

Volatility (vol): How “jumpy” returns are. Practically, volatility is estimated using the standard deviation of recent daily returns.

Sharpe ratio: A common risk-adjusted performance measure. Higher is better. Intuitively, it is “return per unit of risk”.

Drawdown: The percentage fall from a previous peak in the equity curve. Maximum drawdown is the worst such fall.

Rebalancing: Updating portfolio weights on a schedule (for example weekly or monthly).

4 Data and setup

4.1 Environment

I set up:

- VS Code for editing and running code,
- Python (3.14) as the interpreter,
- packages installed using pip: `pandas`, `numpy`, `matplotlib`, `yfinance`.

4.2 Data

Daily adjusted prices were downloaded using `yfinance`. The report window for the final experiments was:

2015-01-01 to 2025-12-26.

This project treats downloaded prices as a practical learning dataset. Data quality and “total return” details are discussed in limitations.

5 Backtesting engine (the core of the project)

5.1 Daily returns

Let P_t be the price on day t . Daily simple return is:

$$r_t = \frac{P_t}{P_{t-1}} - 1.$$

Given a portfolio exposure (weight) $w_t \in [0, 1]$, the portfolio daily return is approximated as:

$$R_t \approx w_{t-1} r_t + (1 - w_{t-1}) r_t^{\text{cash}} - \text{costs}_t,$$

where:

- R_t is the portfolio return on day t .
- w_{t-1} is the fraction of capital invested in risky assets at the end of day $t - 1$.
- r_t is the return of the risky asset (or asset basket) on day t .
- $(1 - w_{t-1})$ is the fraction of capital held in cash.
- $r_{\text{cash},t}$ is the daily return on cash (typically small and often approximated as zero).
- costs_t represents transaction costs incurred on day t due to trading activity.

Example. Suppose $w_{t-1} = 0.8$, meaning 80% of capital is invested and 20% is held in cash. If the risky asset returns $r_t = 1\%$ on day t , cash returns $r_{\text{cash},t} = 0$, and transaction costs are 0.02%, then

$$R_t \approx 0.8 \times 0.01 + 0.2 \times 0 - 0.0002 = 0.0078,$$

corresponding to a portfolio return of 0.78% for the day.

5.2 Costs and cash

This project used simple assumptions:

- **Transaction cost:** 0.0200% per weight change (a simple proxy for spreads/fees).
- **Cash yield:** 3.0% annualised, applied to uninvested capital.

These are not perfect, but they make strategy comparisons more realistic than zero-friction backtests.

5.3 Equity curve

Starting from an initial portfolio value normalised to $E_0 = 1$, the equity curve evolves according to

$$E_t = E_{t-1}(1 + R_t),$$

where:

- E_t is the portfolio value (equity) at the end of day t ,
- E_{t-1} is the portfolio value at the end of the previous day,
- R_t is the portfolio return on day t .

Example. If $E_{t-1} = 1.20$ and the portfolio return on day t is $R_t = -2\%$, then

$$E_t = 1.20 \times (1 - 0.02) = 1.176.$$

5.4 Drawdown

Drawdown at time t is defined as

$$DD_t = \frac{E_t}{\max_{s \leq t} E_s} - 1,$$

where:

- DD_t is the drawdown at time t ,
- E_t is the portfolio value at time t ,
- $\max_{s \leq t} E_s$ is the maximum portfolio value achieved up to time t .

Maximum drawdown is defined as

$$\min_t DD_t,$$

the most negative drawdown observed over the full sample period.

Example. Suppose the portfolio reaches a peak value of 1.50 and later falls to $E_t = 1.20$. The drawdown at time t is

$$DD_t = \frac{1.20}{1.50} - 1 = -20\%.$$

If this is the largest peak-to-trough decline observed, the maximum drawdown of the strategy is -20% .

5.5 Sharpe ratio

Let $\{R_t\}_{t=1}^T$ denote the sequence of daily portfolio returns over the sample period. The annualised Sharpe ratio is approximated as

$$\text{Sharpe} = \frac{\bar{R}}{\sigma(R)}\sqrt{252},$$

where $\bar{R}$ is the mean of daily returns and $\sigma(R)$ is the standard deviation of daily returns.

The mean daily portfolio return is defined as

$$\bar{R} = \frac{1}{T} \sum_{t=1}^T R_t,$$

and the standard deviation of daily returns is given by

$$\sigma(R) = \sqrt{\frac{1}{T-1} \sum_{t=1}^T (R_t - \bar{R})^2},$$

where T is the total number of trading days in the sample. The factor $\sqrt{252}$ is used to annualise the Sharpe ratio, reflecting the approximate number of trading days in a year.

Example. Suppose a backtest contains $T = 2,520$ daily returns (approximately ten years of data). If the mean daily return is $\bar{R} = 0.04\%$ and the standard deviation of daily returns is $\sigma(R) = 0.8\%$, then the annualised Sharpe ratio is

$$\text{Sharpe} = \frac{0.0004}{0.008}\sqrt{252} \approx 0.79.$$

A higher Sharpe ratio indicates a higher average return per unit of risk.

6 Strategy ideas tested

6.1 Strategy 1: Moving average crossover (timing)

A common beginner timing rule is a moving-average crossover:

- compute a fast moving average (example: 20 trading days),
- compute a slow moving average (example: 100 trading days),
- go “risk-on” when fast crosses above slow, else “risk-off”.

This was useful as a learning step, but it generally underperformed buy-and-hold once costs and realistic switching were included.

6.2 Strategy 2: Volatility targeting (position sizing)

Volatility targeting is a position-sizing technique that does not attempt to predict the direction of returns. Instead, it adjusts portfolio exposure in order to maintain a chosen level of risk.

Idea

The central idea is simple:

- when markets are calm (low volatility), portfolio exposure is increased,
- when markets are turbulent (high volatility), portfolio exposure is reduced.

By scaling exposure inversely with realised volatility, the strategy aims to produce a more stable risk profile through time.

Implementation (single-asset exposure)

At each time t , the following steps are applied:

- Estimate realised daily volatility $\sigma_t^{\text{realised}}$ using a rolling window of recent daily returns (for example, the past 20 trading days).
- Choose a target annual volatility level (for example, 10%).
- Convert the annual target volatility to a daily target:

$$\sigma_{\text{daily}}^{\text{target}} = \frac{0.10}{\sqrt{252}}.$$

- Set portfolio exposure according to

$$w_t = \min\left(\text{cap}, \frac{\sigma_{\text{daily}}^{\text{target}}}{\sigma_t^{\text{realised}}}\right),$$

where cap is a maximum exposure constraint (for example, $\text{cap} = 1.0$ to prevent leverage).

The minimum operator ensures that exposure does not exceed the chosen cap, even when realised volatility is very low.

Example (single-asset exposure)

Suppose the target annual volatility is 10%, so that

$$\sigma_{\text{daily}}^{\text{target}} = \frac{0.10}{\sqrt{252}} \approx 0.0063.$$

If the realised daily volatility at time t is $\sigma_t^{\text{realised}} = 1.2\% = 0.012$, then the raw volatility-scaled exposure is

$$\frac{0.0063}{0.012} \approx 0.53.$$

Assuming a cap of 1.0, the portfolio exposure is therefore

$$w_t = \min(1.0, 0.53) = 0.53,$$

meaning that approximately 53% of capital is allocated to risky assets, with the remainder held in cash.

If instead realised volatility were very low (for example 0.4%), the ratio would exceed one, and the cap would bind, resulting in full investment without leverage.

Implementation (two-asset allocation: SPY and QQQ)

The final “best unlevered beat SPY” model in this project uses two risky assets (SPY and QQQ) and applies volatility targeting at the portfolio level. The logic is:

1. Compute each asset’s realised volatility over a rolling window (20 trading days):

$$\sigma_{i,t} = \text{StdDev}(r_{i,t-19}, \dots, r_{i,t}), \quad i \in \{\text{SPY}, \text{QQQ}\}.$$

2. Form base risky weights using inverse-volatility weighting (risk balancing):

$$\tilde{w}_{i,t} = \frac{1/\sigma_{i,t}}{\sum_j 1/\sigma_{j,t}}.$$

This gives lower weight to the more volatile asset and higher weight to the less volatile asset.

3. Estimate portfolio realised volatility from the (lagged) base weights. A simple approximation (ignoring correlations) is:

$$\sigma_t^{\text{port}} \approx \sqrt{\sum_i \tilde{w}_{i,t-1}^2 \sigma_{i,t}^2}.$$

4. Compute a scaling factor to target a chosen annual risk level:

$$s_t = \min\left(\text{cap}, \frac{\sigma_{\text{daily}}^{\text{target}}}{\sigma_t^{\text{port}}}\right), \quad \sigma_{\text{daily}}^{\text{target}} = \frac{0.10}{\sqrt{252}}.$$

5. Final risky weights are the scaled base weights:

$$w_{i,t} = s_t \tilde{w}_{i,t-1}, \quad w_t^{\text{cash}} = 1 - \sum_i w_{i,t}.$$

Using $t - 1$ weights reflects implementability: weights are determined using information available before day t returns are realised.

Example (two-asset allocation)

Assume realised daily volatilities at time t are:

$$\sigma_{\text{SPY},t} = 1.0\% = 0.010, \quad \sigma_{\text{QQQ},t} = 1.6\% = 0.016.$$

The inverse-vol base weights are:

$$\tilde{w}_{\text{SPY},t} = \frac{1/0.010}{1/0.010 + 1/0.016} \approx 0.615, \quad \tilde{w}_{\text{QQQ},t} \approx 0.385.$$

Target daily volatility is:

$$\sigma_{\text{daily}}^{\text{target}} = 0.10/\sqrt{252} \approx 0.0063.$$

Approximate portfolio realised volatility (ignoring correlation) is:

$$\sigma_t^{\text{port}} \approx \sqrt{0.615^2(0.010)^2 + 0.385^2(0.016)^2} \approx 0.0096.$$

So the scaling factor is:

$$s_t = \min(1.0, 0.0063/0.0096) \approx 0.66.$$

Final risky weights become:

$$w_{\text{SPY},t} \approx 0.66 \times 0.615 \approx 0.41, \quad w_{\text{QQQ},t} \approx 0.66 \times 0.385 \approx 0.25,$$

and cash weight is $w_t^{\text{cash}} \approx 1 - (0.41 + 0.25) = 0.34$.

6.3 Strategy 3: Multi-asset allocation and timing variants

I also tested multi-asset universes (examples included equity, bonds, gold). Several timing variants were explored (threshold momentum filters, rotation rules, regime switches). The main learning was that adding layers can reduce drawdowns but does not automatically increase returns, and some combinations can become overly defensive.

7 Headline results

7.1 Benchmarks

Benchmarks used in this project included:

- SPY buy-and-hold (market benchmark),
- QQQ buy-and-hold (growth equity benchmark),
- 50/50 SPY-QQQ buy-and-hold (simple combined benchmark),
- equal-weight baskets (for multi-asset tests).

7.2 Best unlevered “beat SPY” result from this project

The final selected model is a two-asset (SPY, QQQ) volatility-targeted allocation with inverse-vol base weights and a portfolio-level volatility target. Over the report window, it beat SPY buy-and-hold on all three chosen metrics: total return multiple, Sharpe ratio, and maximum drawdown.

Model (2015-01-01 to 2025-12-26)	Final multiple	Sharpe	MaxDD
Vol-target SPY/QQQ (target vol=10%, window=20d, cap=1.00x)	4.45x	1.16	-17.36%
SPY buy-and-hold	4.05x	0.81	-33.72%
QQQ buy-and-hold	6.58x	0.89	-35.12%
50/50 SPY-QQQ buy-and-hold	5.21x	0.87	-30.86%

Table 1: Headline results from the final report window.

Interpretation: The model did not beat QQQ on total return (QQQ had the highest return multiple), but it did beat SPY on all three chosen metrics while remaining unlevered and showing a materially smaller drawdown than SPY and QQQ. It also improved Sharpe and drawdown relative to a naive 50/50 SPY-QQQ buy-and-hold baseline.

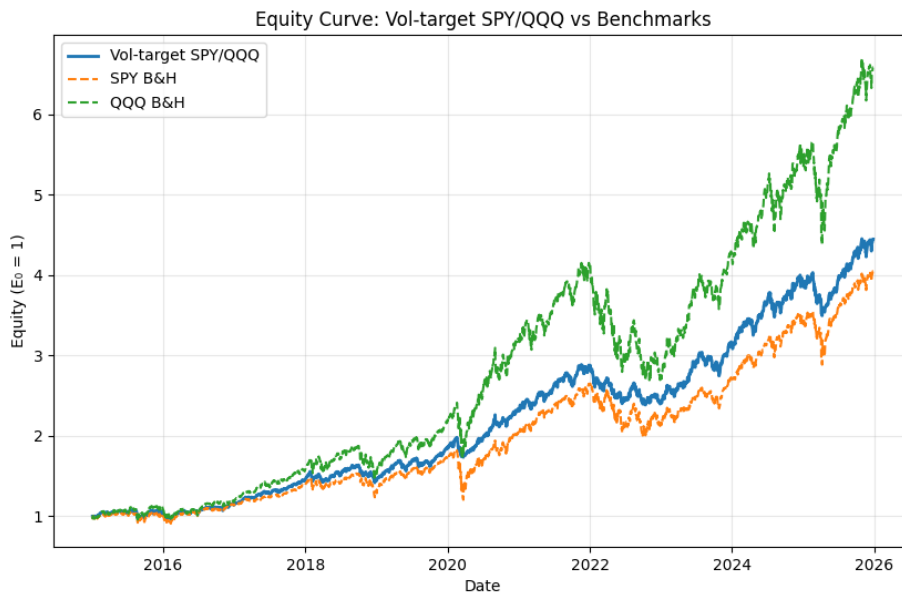


Figure 1: Equity curve: volatility-targeted SPY/QQQ strategy vs SPY buy-and-hold.

8 Key findings and practical takeaways

8.1 Simple timing rules are not automatically profitable

A key empirical takeaway is that many intuitive timing rules that look compelling on charts (for example moving-average crossovers) can underperform buy-and-hold once basic frictions are introduced, including transaction costs, turnover, and implementation lag. This highlights the importance of testing strategies under realistic assumptions rather than relying on visual pattern-matching.

8.2 Risk management can matter more than forecasting

Across experiments, position sizing and risk control were more consistent sources of improvement than attempting to predict direction. Volatility targeting in particular provides a systematic mechanism for reducing exposure in high-volatility periods and increasing exposure in calmer regimes, which can improve risk-adjusted performance and reduce drawdowns even when raw returns are similar.

8.3 Complexity must justify itself

Adding more assets, filters, or regime logic did not reliably improve outcomes. Extra layers can reduce drawdowns but may also dilute returns, increase turnover, and introduce parameter sensitivity. A useful workflow is to treat each added component as a hypothesis that must earn its place via robustness checks and out-of-sample validation.

9 Limitations

This is a learning project, not investment advice. Limitations include:

- **Data source:** `yfinance` is convenient but not institutional-grade.
- **Total return:** dividends and corporate actions may not be handled perfectly depending on the series used (“Adj Close” vs “Close”, and vendor conventions).
- **Costs model:** transaction costs are simplified and do not include slippage or spread dynamics.
- **Portfolio risk estimate:** the portfolio volatility estimate used is a simple approximation and does not explicitly model correlations.
- **Overfitting risk:** parameter sweeps and repeated iterations can select lucky configurations. Robustness checks help but do not eliminate this risk.

10 Future work and improvement roadmap

Planned improvements:

- UK realism: run the same framework using UCITS ETFs available to UK investors.
- Robustness: walk-forward testing and multiple train-test splits with fixed rules.
- Better execution realism: slippage model, trading calendar handling, and position rounding.
- Live paper trading: run the strategy daily with logs and a weekly report.

A Minimal pseudo-code for the research pipeline

```
1 1) Download ETF prices
2 2) Compute daily returns
3 3) Estimate realised volatility (rolling window)
4 4) Compute base weights (e.g., inverse-vol across assets)
5 5) Estimate portfolio realised volatility
6 6) Compute scaler to hit target risk; apply exposure cap
7 7) Apply cash allocation and costs from turnover
8 8) Build equity curve
9 9) Report: final multiple, Sharpe, max drawdown, turnover
```

B Python code used for the final SPY/QQQ volatility-target strategy

The code below is included for reproducibility. It downloads prices, runs the final two-asset volatility-target model, prints the headline metrics, and saves the equity curve figure as `equity space curve.png`.

```
1 import numpy as np
2 import pandas as pd
3 import yfinance as yf
4 import matplotlib.pyplot as plt
5
6 def download_prices(tickers, start):
7     df = yf.download(tickers, start=start, progress=False, auto_adjust=
8         False)
9     if df is None or df.empty:
10         raise RuntimeError("No data returned by yfinance.")
11
12     # MultiIndex: (field, ticker)
13     if isinstance(df.columns, pd.MultiIndex):
14         fields = df.columns.get_level_values(0).unique().tolist()
15         field = "Adj Close" if "Adj Close" in fields else ("Close" if "
16             Close" in fields else None)
17         if field is None:
18             raise KeyError(f"No Adj Close / Close. Fields: {fields}")
19         px = df[field].copy()
20     else:
21         cols = df.columns.tolist()
22         if "Adj Close" in cols:
23             px = df[["Adj Close"]].copy()
24         elif "Close" in cols:
25             px = df[["Close"]].copy()
26         else:
27             raise KeyError(f"No Adj Close / Close. Columns: {cols}")
28         if isinstance(tickers, (list, tuple)) and len(tickers) == 1:
29             px.columns = [tickers[0]]
30         elif isinstance(tickers, str):
31             px.columns = [tickers]
32
33     tick_list = list(tickers) if not isinstance(tickers, str) else [
34         tickers]
35     px = px.loc[:, [c for c in tick_list if c in px.columns]].dropna(
36         how="all")
37     px = px.dropna()
```

```

34     return px
35
36 def sharpe(daily_ret):
37     x = pd.Series(daily_ret).dropna()
38     s = x.std(ddof=1)
39     if s == 0:
40         return np.nan
41     return float((x.mean() / s) * np.sqrt(252))
42
43 def max_dd(eq):
44     eq = pd.Series(eq).dropna()
45     peak = eq.cummax()
46     dd = eq / peak - 1.0
47     return float(dd.min())
48
49 def run_vol_target_two_asset(
50     tickers=("SPY", "QQQ"),
51     start_date="2015-01-01",
52     vol_window=20,
53     target_ann_vol=0.10,
54     cap=1.0,
55     cash_rate=0.03,
56     cost_rate=0.0002
57 ):
58     px = download_prices(tickers, start_date)
59     ret = px.pct_change().dropna()
60
61     # realised vol per asset (daily)
62     vol = ret.rolling(vol_window).std()
63
64     # inverse-vol base weights
65     inv = 1.0 / vol
66     base_w = inv.div(inv.sum(axis=1), axis=0)
67
68     # lag weights for implementability
69     base_w_lag = base_w.shift(1)
70
71     # portfolio realised vol (simple approximation, ignoring
72     # correlation)
73     port_var = (base_w_lag.pow(2) * vol.pow(2)).sum(axis=1)
74     port_vol = np.sqrt(port_var)
75
76     # scale exposure to target portfolio vol
77     target_daily_vol = target_ann_vol / np.sqrt(252)
78     scale = (target_daily_vol / port_vol).clip(upper=cap).fillna(0.0)
79
80     # final risky weights and cash weight
81     w_risky = (base_w_lag.mul(scale, axis=0)).fillna(0.0)
82     w_cash = 1.0 - w_risky.sum(axis=1)
83
84     # turnover and costs
85     turnover = w_risky.diff().abs().sum(axis=1).fillna(0.0)
86     costs = turnover * cost_rate
87
88     cash_daily = cash_rate / 252.0
89
90     # portfolio return and equity
91     port_ret = (w_risky * ret).sum(axis=1) + w_cash * cash_daily -

```

```

costs
equity = (1.0 + port_ret).cumprod()

# benchmarks
spy_bh = (1.0 + ret["SPY"]).cumprod()
qqq_bh = (1.0 + ret["QQQ"]).cumprod()
fifty = (1.0 + 0.5*ret["SPY"] + 0.5*ret["QQQ"]).cumprod()

print(f"VOL-TARGET (2-asset) {list(tickers)} | target_vol={
    target_ann_vol:.0%} | vw={vol_window} | cap={cap:.2f}x")
print(f"Window: {equity.index.min().date()} -> {equity.index.max().
    date()}")
print(f"STRATEGY final={equity.iloc[-1]:.2f}x | sharpe={sharpe(
    port_ret):.2f} | maxDD={max_dd(equity):.2%}")
print(f"SPY B&H final={spy_bh.iloc[-1]:.2f}x | sharpe={sharpe(ret
    ['SPY']):.2f} | maxDD={max_dd(spy_bh):.2%}")
print(f"QQQ B&H final={qqq_bh.iloc[-1]:.2f}x | sharpe={sharpe(ret
    ['QQQ']):.2f} | maxDD={max_dd(qqq_bh):.2%}")
print(f"50/50 final={fifty.iloc[-1]:.2f}x | sharpe={sharpe(0.5*
    ret['SPY']+0.5*ret['QQQ']):.2f} | maxDD={max_dd(fifty):.2%}")

# save figure with the exact filename used in the paper
plt.figure(figsize=(10,6))
plt.plot(equity, label="Vol-target SPY/QQQ", linewidth=2)
plt.plot(spy_bh, label="SPY Buy & Hold", linestyle="--")
plt.title("Equity Curve: Volatility Targeting vs SPY")
plt.xlabel("Date")
plt.ylabel("Equity (EO = 1)")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("equity space curve.png", dpi=300, bbox_inches="tight")
plt.show()

if __name__ == "__main__":
    run_vol_target_two_asset()

```